Gold Layer: Step-by-Step Guide (AdventureWorks on Databricks)

> **Where you are:** Bronze ✅ → Silver ✅ → **Gold (this guide)** → Semantic Layer → Dashboard

---

Step 0: What is the Gold layer, in plain words?

- **Silver** = clean data. Each table is correct, but it still looks like the source files.
- **Gold** = data arranged so business questions are **easy** to answer.

In Gold, you arrange data into a **star schema.** It has just two kinds of tables.

Type	What it holds	Example	Think of it as
Fact	Things that *happened*, with numbers	A sale, a return	"The events"
Dimension	Descriptions you filter or group by	Customer, Product, Date, Territory	"Who / What / Where / When"

The fact table sits in the middle. The dimensions sit around it, like points of a star.

```
                dim_customer
                     |
dim_date ---- fact_sales ---- dim_product
                     |
                dim_territory

dim_date ---- fact_returns ---- dim_product
                     |
                dim_territory
```

What you will build (6 tables)

#	Table	One row =
1	`gold_dim_date`	one calendar day
2	`gold_dim_customer`	one customer
3	`gold_dim_product`	one product (with its subcategory and category included)
4	`gold_dim_territory`	one sales territory
5	`gold_fact_sales`	one order line
6	`gold_fact_returns`	one return record

> **Note:** The original guide mentions "5 dimension tables", but it lists only 4. That is because **Subcategory** and **Category** are merged *into* `dim_product`. They do not become separate tables. So you get **4 dimensions + 2 facts**.

> **Also note:** The 3 sales years were already combined in Silver. You do **not** need to union them again in Gold.

Build order matters: build the dimensions first, then the facts. Facts point to dimensions, so the dimensions must exist first.

Step 1: Create a place for Gold tables

Run this once in a SQL cell. Change `adventureworks` to your catalog name.

```sql
CREATE SCHEMA IF NOT EXISTS adventureworks.gold;
```

> If you save by ADLS path instead of Unity Catalog, use `.save("abfss://gold@<storage>.dfs.core.windows.net/<table>")` in place of `.saveAsTable(...)`.

Step 2: Load your Silver tables

Create one notebook (for example `gold_build`) and start by reading Silver.

```python
```

```python
from pyspark.sql.functions import (
    col, when, concat_ws, date_format, floor, months_between,
    current_date, round as spark_round, year, quarter, month,
    dayofweek, weekofyear
)

SILVER = "adventureworks.silver"
GOLD   = "adventureworks.gold"

sales         = spark.table(f"{SILVER}.sales")
returns       = spark.table(f"{SILVER}.returns")
customers     = spark.table(f"{SILVER}.customers")
products      = spark.table(f"{SILVER}.products")
subcategories = spark.table(f"{SILVER}.product_subcategories")
categories    = spark.table(f"{SILVER}.product_categories")
territories   = spark.table(f"{SILVER}.territories")
calendar      = spark.table(f"{SILVER}.calendar")
```

Change the table names to match what you called them in Silver.

Step 3: Build `gold_dim_date`

Why: It lets you slice any number by year, quarter, month, or weekday.

What we add: a `DateKey` number like `20170115`. Joining on a number is fast and simple.

python

```python
dim_date = (calendar
    .withColumn("DateKey",    date_format("Date", "yyyyMMdd").cast("int"))
    .withColumn("Year",       year("Date"))
    .withColumn("Quarter",    quarter("Date"))
    .withColumn("Month",      month("Date"))
    .withColumn("MonthName",  date_format("Date", "MMMM"))
    .withColumn("YearMonth",  date_format("Date", "yyyy-MM"))
    .withColumn("DayName",    date_format("Date", "EEEE"))
    .withColumn("WeekOfYear", weekofyear("Date"))
    .withColumn("IsWeekend",  dayofweek("Date").isin(1, 7))   # 1 = Sunday, 7 =
    .select("DateKey", "Date", "Year", "Quarter", "Month",
            "MonthName", "YearMonth", "DayName", "WeekOfYear", "IsWeekend")
)

dim_date.write.format("delta").mode("overwrite").saveAsTable(f"{GOLD}.gold_dim_
```

If you already added some of these columns in Silver, just keep the ones you need.

Step 4: Build `gold_dim_customer`

Why: It answers "who is buying?"

What we add:

- `FullName`: first and last name together
- `Age`: calculated from `BirthDate`
- `AgeGroup`: buckets that are easy to chart

```python
dim_customer = (customers
    .withColumn("FullName", concat_ws(" ", col("FirstName"), col("LastName")))
    .withColumn("Age", floor(months_between(current_date(), col("BirthDate")) /
    .withColumn("AgeGroup",
        when(col("Age") < 30, "Under 30")
        .when(col("Age") < 45, "30-44")
        .when(col("Age") < 60, "45-59")
        .otherwise("60+"))
    .select("CustomerKey", "FullName", "FirstName", "LastName", "Gender",
            "BirthDate", "Age", "AgeGroup", "MaritalStatus", "AnnualIncome",
            "TotalChildren", "EducationLevel", "Occupation", "HomeOwner")
)

dim_customer.write.format("delta").mode("overwrite").saveAsTable(f"{GOLD}.gold_
```

> **Tip:** The AdventureWorks data is old, so ages based on *today* will look high. That is fine for learning. In real projects, you may calculate age at the order date instead.

Step 5: Build `gold_dim_product`

Why: It answers "what is being sold?"

The key idea: In Silver, product info is spread over 3 tables:

```
Products  →  Subcategories  →  Categories
```

In Gold, we **join them into one table**, so users never need to do that join.

```python
```

```
dim_product = (products
    .join(subcategories, "ProductSubcategoryKey", "left")
    .join(categories,    "ProductCategoryKey",    "left")
    .withColumn("MarginPct",
        spark_round((col("ProductPrice") - col("ProductCost")) / col("ProductPr
    .select("ProductKey", "ProductSKU", "ProductName", "ModelName",
            "ProductColor", "ProductSize", "ProductStyle",
            "SubcategoryName", "CategoryName",
            "ProductCost", "ProductPrice", "MarginPct")
)

dim_product.write.format("delta").mode("overwrite").saveAsTable(f"{GOLD}.gold_d
```

Why a `left` join? If a product has no subcategory, it still stays in the table. An `inner` join would silently drop it.

Step 6: Build `gold_dim_territory`

Why: It answers "where is it sold?"

The source column is `SalesTerritoryKey`, but Sales and Returns call it `TerritoryKey`. We rename it, so the names match everywhere.

```
python

dim_territory = (territories
    .withColumnRenamed("SalesTerritoryKey", "TerritoryKey")
    .select("TerritoryKey", "Region", "Country", "Continent")
)

dim_territory.write.format("delta").mode("overwrite").saveAsTable(f"{GOLD}.gold
```

Step 7: Build `gold_fact_sales` (the most important table)

Why: This is where revenue and profit live.

The problem: Sales rows only have `ProductKey` and `OrderQuantity`. They have no price.

The fix: Join to Products to get the price and cost, then do the math.

New column	Formula
`ExtendedRevenue`	Quantity × Price
`ExtendedCost`	Quantity × Cost
`GrossProfit`	Revenue − Cost

We also turn the dates into `DateKey` numbers, so they match `dim_date`.

python

```
price_lookup = products.select("ProductKey", "ProductPrice", "ProductCost")

fact_sales = (sales
    .join(price_lookup, "ProductKey", "left")
    .withColumn("OrderDateKey", date_format("OrderDate", "yyyyMMdd").cast("int"
    .withColumn("StockDateKey", date_format("StockDate", "yyyyMMdd").cast("int"
    .withColumn("UnitPrice",       col("ProductPrice"))
    .withColumn("UnitCost",        col("ProductCost"))
    .withColumn("ExtendedRevenue", col("OrderQuantity") * col("ProductPrice"))
    .withColumn("ExtendedCost",    col("OrderQuantity") * col("ProductCost"))
    .withColumn("GrossProfit",     col("ExtendedRevenue") - col("ExtendedCost")
    .withColumn("OrderYear",       year("OrderDate"))
    .select(
        # keys (links to dimensions)
        "SalesOrderLineKey", "OrderNumber", "OrderLineItem",
        "OrderDateKey", "StockDateKey",
        "CustomerKey", "ProductKey", "TerritoryKey",
        # measures (the numbers)
        "OrderQuantity", "UnitPrice", "UnitCost",
        "ExtendedRevenue", "ExtendedCost", "GrossProfit",
        # for partitioning
        "OrderYear"
    )
)

(fact_sales.write.format("delta")
    .mode("overwrite")
    .partitionBy("OrderYear")
    .saveAsTable(f"{GOLD}.gold_fact_sales"))
```

> **Good to know:** The Products table has one price per product (today's price). This means all years use the same price. That is normal for this dataset. In real systems, the price is

> usually stored on the order line itself.

Step 8: Build `gold_fact_returns`

Why: It answers "what came back?"

It works the same way as sales, just simpler.

python

```
fact_returns = (returns
    .join(price_lookup, "ProductKey", "left")
    .withColumn("ReturnDateKey", date_format("ReturnDate", "yyyyMMdd").cast("in
    .withColumn("ReturnValue",   col("ReturnQuantity") * col("ProductPrice"))
    .select("ReturnDateKey", "ProductKey", "TerritoryKey",
            "ReturnQuantity", "ReturnValue")
)

fact_returns.write.format("delta").mode("overwrite").saveAsTable(f"{GOLD}.gold_
```

> Returns have **no** `CustomerKey` or order number in this dataset. So returns connect to Product, Territory, and Date only.

Step 9: Check your work (do not skip this)

Bad joins give wrong numbers **without any error.** These checks catch that.

9a. Row counts should match

`fact_sales` must have the same number of rows as Silver sales. If it has more, a join created duplicates.

python

```
print("Silver sales :", sales.count())
print("Gold sales   :", spark.table(f"{GOLD}.gold_fact_sales").count())

print("Silver returns:", returns.count())
print("Gold returns  :", spark.table(f"{GOLD}.gold_fact_returns").count())
```

9b. No "orphan" keys

Every key in a fact table must exist in its dimension. Each result below should be **0.**

python

```
fs = spark.table(f"{GOLD}.gold_fact_sales")

checks = {
    "Missing products":    fs.join(dim_product,   "ProductKey",   "left_anti"),
    "Missing customers":   fs.join(dim_customer,  "CustomerKey",  "left_anti"),
    "Missing territories": fs.join(dim_territory, "TerritoryKey", "left_anti"),
    "Missing order dates": fs.join(dim_date.withColumnRenamed("DateKey", "Order
                                   "OrderDateKey", "left_anti"),
}

for name, df in checks.items():
    print(name, ":", df.count())
```

9c. No missing money values

python

```
print("Null revenue rows:", fs.filter(col("ExtendedRevenue").isNull()).count())
```

9d. Keys are unique in dimensions

python

```
print(dim_product.count(),  dim_product.select("ProductKey").distinct().count()
print(dim_customer.count(), dim_customer.select("CustomerKey").distinct().count
```

Step 10: Try some business questions

If these queries run and the numbers look sensible, your Gold layer works.

Revenue and profit by year

sql

```
SELECT d.Year,
       SUM(f.ExtendedRevenue) AS Revenue,
       SUM(f.GrossProfit)     AS Profit
FROM adventureworks.gold.gold_fact_sales f
JOIN adventureworks.gold.gold_dim_date d ON f.OrderDateKey = d.DateKey
GROUP BY d.Year
ORDER BY d.Year;
```

Revenue by category

```sql
SELECT p.CategoryName, SUM(f.ExtendedRevenue) AS Revenue
FROM adventureworks.gold.gold_fact_sales f
JOIN adventureworks.gold.gold_dim_product p ON f.ProductKey = p.ProductKey
GROUP BY p.CategoryName
ORDER BY Revenue DESC;
```

Return rate by category

```sql
WITH sold AS (
  SELECT ProductKey, SUM(OrderQuantity) AS QtySold
  FROM adventureworks.gold.gold_fact_sales GROUP BY ProductKey
),
returned AS (
  SELECT ProductKey, SUM(ReturnQuantity) AS QtyReturned
  FROM adventureworks.gold.gold_fact_returns GROUP BY ProductKey
)
SELECT p.CategoryName,
       ROUND(SUM(COALESCE(r.QtyReturned, 0)) / SUM(s.QtySold) * 100, 2) AS Retu
FROM sold s
JOIN adventureworks.gold.gold_dim_product p ON s.ProductKey = p.ProductKey
LEFT JOIN returned r ON s.ProductKey = r.ProductKey
GROUP BY p.CategoryName;
```

Top 10 customers

```sql
SELECT c.FullName, SUM(f.ExtendedRevenue) AS Revenue
FROM adventureworks.gold.gold_fact_sales f
JOIN adventureworks.gold.gold_dim_customer c ON f.CustomerKey = c.CustomerKey
GROUP BY c.FullName
ORDER BY Revenue DESC
LIMIT 10;
```

> Notice the return rate query: the two fact tables are **never joined directly**. Each one is summed first, and then they meet through a shared dimension (Product). Joining two fact tables directly multiplies rows and gives wrong totals.

Quick summary

Step	What you do	Result
1	Create the `gold` schema	A home for Gold tables
2	Read the Silver tables	Inputs ready
3	Add date parts and `DateKey`	`gold_dim_date`
4	Add FullName, Age, AgeGroup	`gold_dim_customer`
5	Join Product + Subcategory + Category	`gold_dim_product`
6	Rename key to `TerritoryKey`	`gold_dim_territory`
7	Join price, compute Revenue, Cost, Profit	`gold_fact_sales`
8	Join price, compute ReturnValue	`gold_fact_returns`
9	Row counts, orphan keys, nulls, uniqueness	Trustworthy data
10	Run business queries	Proof it works

Three rules to remember

1. **Dimensions first, facts second.**
2. **Fact row count must equal the Silver row count.** If not, a join is wrong.
3. **Every fact key must exist in its dimension.** Orphan count should be 0.

What comes next

1. **Semantic layer:** Define Revenue, Gross Profit, Units Sold, and Return Rate once (for example as Unity Catalog Metric Views), so everyone uses the same formula.
2. **Dashboard v1:** Build a Databricks SQL dashboard using the queries from Step 10.
3. **Automate:** Put the Bronze → Silver → Gold notebooks into one Databricks Job and schedule it.